Module 5 - Fundamental Data Structures - Stacks and Queues

1.0 Learning Outcomes

At the end of this week/topic, the students should be able to:

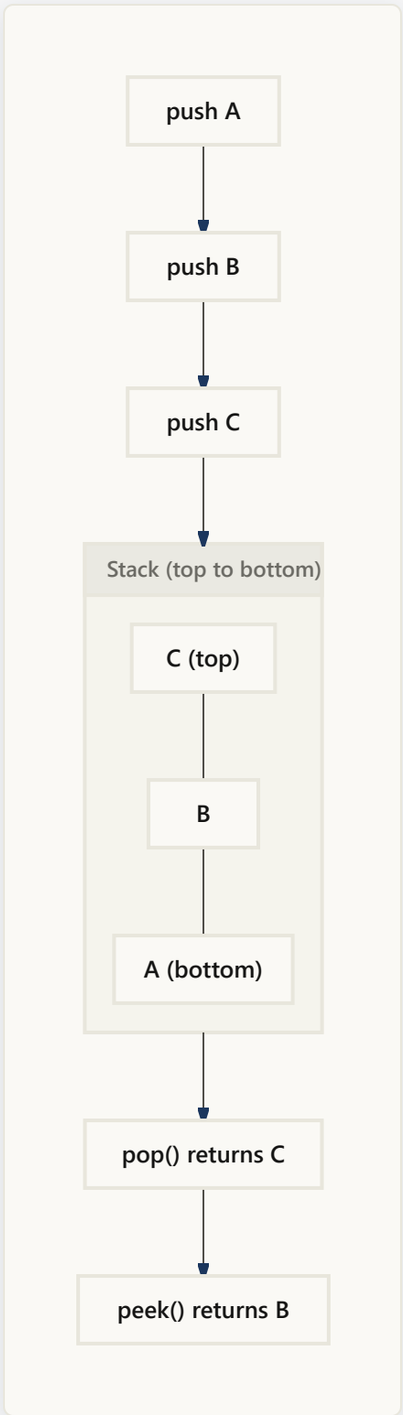
1. **Explain** the LIFO principle and **implement** stack operations (push, pop, peek).
2. **Explain** the FIFO principle and **implement** queue operations (enqueue, dequeue).
3. **Implement** stacks and queues using both **arrays** and **linked lists**.
4. **Describe** real-world applications of stacks (function calls, undo operations).
5. **Describe** real-world applications of queues (task scheduling, buffering).

2.0 The Stack

2.1 The LIFO Principle

A **stack** is a linear data structure in which insertion and removal happen at only **one end**, called the **top**. The defining rule is **LIFO (Last In, First Out)**: the most recently added element is the first one removed.

Analogy: a stack of plates in a cafeteria. You place a new plate on top, and you can only take the top plate. To reach the bottom plate, you must remove every plate above it first.



2.2 Stack Operations

Operation	Description	Precondition	Time complexity
`push(x)`	Insert `x` at the top	Stack not full (array-based)	O(1)
`pop()`	Remove and return the top element	Stack not empty	O(1)
`peek()`	Return the top element without removing it	Stack not empty	O(1)
`is_empty()`	Return true if the stack has no elements	None	O(1)
`is_full()`	Return true if the array stack is at capacity	Array-based only	O(1)

Two error conditions must always be handled explicitly:

- **Stack underflow**: `pop()` or `peek()` on an empty stack.
- **Stack overflow**: `push()` on a full fixed-size array.

2.3 Array Implementation

Idea: store elements in a fixed-size Python list; an integer `top` tracks the index of the current top element. Start with `top = -1` (empty). `push` increments `top` then stores; `pop` reads then decrements.

```
class ArrayStack:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.top = -1

    def is_empty(self):
        return self.top == -1

    def is_full(self):
        return self.top == len(self.data) - 1

    def push(self, value):
        if self.is_full():
            raise OverflowError("Stack overflow")
        self.top = self.top + 1
        self.data[self.top] = value

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack underflow")
        value = self.data[self.top]   # read the top value first
        self.top = self.top - 1      # then move top down
        return value

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.data[self.top]
```

2.4 Linked List Implementation

Idea: push and pop at the **head** of a singly linked list, so both are O(1) and no overflow occurs until memory runs out. The `Node` class below is shared with the linked queue in Section 3.4.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None


class LinkedStack:
    def __init__(self):
        self.top = None    # head of the list = top of the stack

    def is_empty(self):
        return self.top is None
```

```
    def push(self, value):
        node = Node(value)
        node.next = self.top    # new node points to old top
        self.top = node         # new node becomes the top

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack underflow")
        value = self.top.value
        self.top = self.top.next    # move top down the chain
        return value

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.top.value
```

> **Key point for students:** inserting or removing at the tail of a singly linked list would make `pop` O(n). Always operate at the head.

2.5 Applications of Stacks

1. Function calls (the *call stack*).
Every function call creates an **activation record (stack frame)** holding parameters, local variables, and the return address. Frames are pushed on call and popped on return. This is why local variables vanish when a function ends, and why deep or infinite recursion raises a `RecursionError` in Python: the call stack ran out of space.

```
def factorial(n):                # factorial(3)
    if n <= 1:
        return 1                 # pushes factorial(3), factorial(2),
factorial(1)
    return n * factorial(n - 1)  # pops in reverse: 1, then 2*1, then 3*2
```

2. Undo operations.
Editors keep completed actions on a stack; **Undo** pops the last action and reverses it. **Redo** is a second stack: undoing pushes onto the redo stack, and any new action clears it.

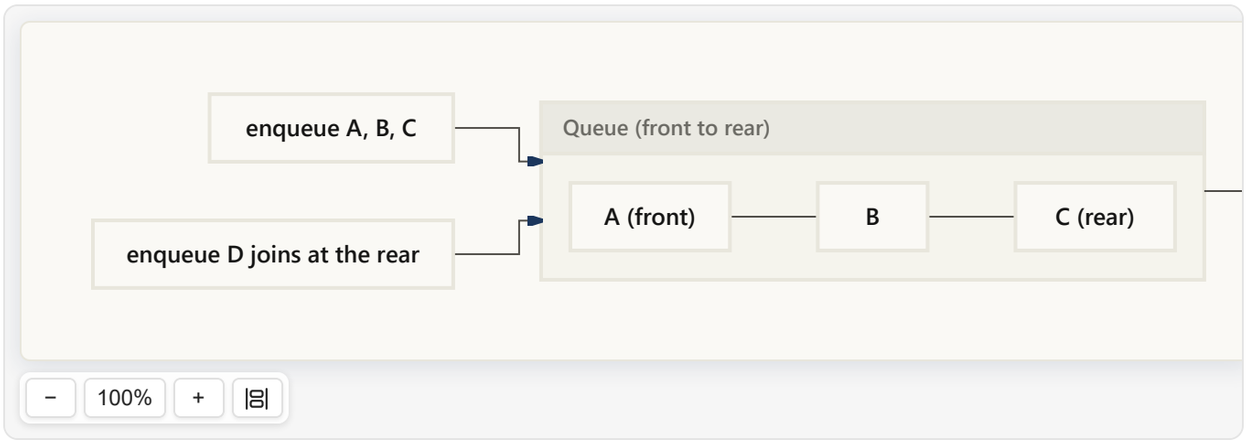
3. Other applications to mention: balanced-parentheses checking, infix-to-postfix conversion and expression evaluation, browser back-button history, depth-first search (DFS).

3.0 The Queue

3.1 The FIFO Principle

A **queue** is a linear data structure with insertion at one end (**rear** or **tail**) and removal at the other end (**front** or **head**). Its rule is **FIFO (First In, First Out)**: the element that has waited longest is removed first.

Analogy: a line at a cashier. Newcomers join at the back; the person served is always the one at the front. No cutting.



3.2 Queue Operations

Operation	Description	Precondition	Time complexity
`enqueue(x)`	Insert `x` at the rear	Queue not full (array-based)	O(1)
`dequeue()`	Remove and return the front element	Queue not empty	O(1)
`peek_front()`	Return the front element without removing it	Queue not empty	O(1)
`is_empty()`	Return true if the queue has no elements	None	O(1)
`is_full()`	Return true if the array queue is at capacity	Array-based only	O(1)

Error conditions: **underflow** (`dequeue` on an empty queue) and **overflow** (`enqueue` on a full fixed array).

3.3 Array Implementation: the Circular Queue

Problem with a naive array: if the front keeps moving forward, or if we shift elements left after every dequeue, we either waste space at the front or make `dequeue` O(n).

S*olution*: treat the array as **circular** using the modulo operator. Both `front` and `rear` indices wrap around to index 0 when they pass the end, so empty slots at the front become reusable.

```
class ArrayQueue:
    def __init__(self, capacity):
        self.data = [None] * capacity
        self.front = 0
        self.size = 0

    def is_empty(self):
        return self.size == 0

    def is_full(self):
        return self.size == len(self.data)

    def enqueue(self, value):
        if self.is_full():
            raise OverflowError("Queue overflow")
        rear = (self.front + self.size) % len(self.data)    # wrap around
        self.data[rear] = value
```

```
        self.size = self.size + 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue underflow")
        value = self.data[self.front]
        self.front = (self.front + 1) % len(self.data)     # wrap around
        self.size = self.size - 1
        return value

    def peek_front(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.data[self.front]
```

3.4 Linked List Implementation

Idea: keep a reference to **both** the head (front, for dequeue) and the tail (rear, for enqueue), so both operations are O(1). This reuses the `Node` class from Section 2.4.

```
class LinkedQueue:
    def __init__(self):
        self.front = None    # dequeue from the head
        self.rear = None     # enqueue at the tail

    def is_empty(self):
        return self.front is None

    def enqueue(self, value):
        node = Node(value)
        if self.is_empty():
            self.front = node        # first element is both front and rear
        else:
            self.rear.next = node    # link after the old tail
        self.rear = node             # new node becomes the rear

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue underflow")
        value = self.front.value
        self.front = self.front.next
        if self.front is None:        # queue became empty
            self.rear = None
        return value

    def peek_front(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.front.value
```

Variants worth mentioning: deque (insert and remove at both ends, `collections.deque` in Python), priority queue (highest priority leaves first, not oldest), circular queue (ring buffer).

3.5 Applications of Queues

1. Task scheduling.
The OS ready queue holds processes waiting for the CPU; the scheduler serves them in arrival

order (round-robin). Print spoolers queue documents in the order submitted: the first job in prints first.

2. Buffering.
Buffers smooth out speed differences between producer and consumer: a keyboard buffer queues keystrokes faster than the application consumes them; audio and video streaming buffers packets so playback stays smooth; I/O requests to a disk or network interface are queued.

3. Other applications to mention: breadth-first search (BFS), call-center or customer-service ticketing systems, message queues in distributed systems.

4.0 Stacks vs. Queues at a Glance

Aspect	Stack	Queue
Principle	LIFO (last in, first out)	FIFO (first in, first out)
Insert and remove ends	Same end (top)	Opposite ends (rear / front)
Core operations	push, pop, peek	enqueue, dequeue, peek_front
Array implementation	`top` index	Circular buffer with `front` + `size`
Linked list anchor	Head only	Head and tail references
Typical applications	Function calls, undo, DFS, expression evaluation	Scheduling, buffering, BFS

5.0 Laboratory Activity

Lab title: Stacks and Queues: Array vs. Linked List
Objectives: map directly to Learning Outcomes 1 to 3.

5.1 Instructions

1. Implement `ArrayStack` (push, pop, peek, is_empty, is_full) exactly as specified, including underflow and overflow checks that raise exceptions with descriptive messages.
2. Implement `LinkedStack` with the same operations (no fixed capacity).
3. Implement the circular `ArrayQueue` (enqueue, dequeue, peek_front, is_empty, is_full). Demonstrate wrap-around by enqueuing past the physical end of the array.
4. Implement `LinkedQueue` with head and tail references.
5. Write a menu-driven program that lets the user choose the implementation and perform any operation, printing the state of the structure after each operation.
6. **Application exercise (pair work):**
   - a. Use a stack to check whether a string of parentheses `()[]{}` is balanced.
   - b. Simulate a print queue: enqueue jobs with IDs and page counts, dequeue and print them in FIFO order, reporting wait order.
7. **Analysis question:** briefly explain (3 to 5 sentences) why the linked-list stack pushes at the head instead of the tail, and why the array queue must be circular.

5.2 Deliverables

- Source files for the four classes and the test driver, properly commented.
- Sample runs showing error handling (underflow and overflow).

- Written answer to the analysis question, saved as `analysis.md`.
- All files uploaded individually and unzipped (no archive file) to a GitHub repository under the GitHub account you will create for this course named Activity#_Section_LASTNAME_FIRSTNAME. The documentation must be written in Markdown: a `README.md` describing the program, its files, and how to run each implementation.
- The repository link submitted to the LMS before the deadline.

7.0 Common Implementation Mistakes to Watch For

- Off-by-one errors: initializing `top` to `0` instead of `-1`, or updating the index before or after accessing the element in the wrong order.
- Forgetting to update `rear` (or `front`) to `None` when the linked queue becomes empty, which causes bad enqueues later.
- Using `front == rear` to detect fullness in a circular queue (ambiguous between empty and full); use a `size` counter.
- Popping from the tail of a singly linked list (O(n)) instead of the head.
- Silently ignoring underflow and overflow instead of raising an exception or reporting an error.
- Misusing Python lists as stacks or queues with `insert(0, x)` or `pop(0)`, which are O(n); the point of the lab is to build the structures from first principles.